

Documentación del Proyecto:

Calculadora Visual e Interactiva de Circuitos Resistivos

Autor: Grupo "Juiciosos"

Versión: 1.0 | **Fecha:** 2026

1 ¿Qué problema se intenta resolver?

El análisis de circuitos eléctricos mediante métodos teóricos manuales (como el Análisis de Mallas basado en las Leyes de Kirchhoff) es un proceso propenso a errores humanos en varios niveles:

- Identificación topológica de mallas y nodos:** Errores al identificar nodos esenciales, simples y recorridos cerrados independientes.
- Planteamiento e inversión de sistemas de ecuaciones linealmente independientes:** Dificultad para convertir restricciones del circuito en matrices algebraicas correctas.
- Falta de retroalimentación visual directa:** Imposibilidad de validar interactivamente los resultados teóricos (caídas de tensión, sentido de la corriente, disipación de potencia) elemento por elemento.

Objetivo de la Solución: Desarrollar una aplicación de escritorio multiplataforma rápida, optimizada y de bajo tiempo de desarrollo utilizando el ecosistema **Python**, que permita diseñar de forma interactiva circuitos eléctricos de hasta **20 resistencias** y **3 fuentes de voltaje DC**. La aplicación debe detectar de forma automática nodos (simples y esenciales), determinar inductivamente las mallas independientes tomando como pivote la fuente de mayor magnitud, resolver las corrientes de malla mediante álgebra lineal, y presentar dinámicamente las matrices resultantes junto con la inspección individual de componentes.

2 Selección de Tecnologías (Ecosistema Python Full-Stack)

Para garantizar un tiempo de desarrollo acelerado, rendimiento nativo fluido, portabilidad sin dependencias externas complejas y alta usabilidad/accesibilidad, se selecciona la siguiente pila tecnológica:

- Lenguaje Principal: Python 3.11+** (por su expresividad, madurez en cálculo numérico y soporte gráfico robusto).

- **Interfaz Gráfica de Usuario (GUI) y Lienzo Interactivo: PyQt6 (o PySide6).**
 - *Razón de elección:* Proporciona el módulo `QGraphicsView / QGraphicsScene`, ideal para renderizado vectorial acelerado de esquemáticos, manejo de eventos de clic en componentes (resistencias/fuentes) y arrastre de elementos. Requiere menor esfuerzo y tiempo de desarrollo que librerías web compiladas, manteniendo ejecución nativa.
- **Motor Matemático y Álgebra Lineal: NumPy y SymPy.**
 - NumPy: Inversión de matrices, producto vectorial/matricial y resolución numérica ultra rápida ($Ax = b$).
 - SymPy (*Opcional*): Formateo simbólico de ecuaciones matriciales.
- **Estructura de Grafos Topológicos: NetworkX.**
 - Permite modelar el circuito como un grafo direccional/no-direccional ponderado, facilitando la extracción automática de ciclos independientes (mallas) y cálculo de grados de nodos.
- **Formateo de Matemáticas y Accesibilidad: Matplotlib** (módulo `mathtext` integrado con PyQt) para renderizar las matrices matemáticas con notación rigurosa $\LaTeX$.

3 Descripción Detallada del Paso a Paso para Resolver el Problema

Paso 1: Representación Topológica del Circuito (Grafo)

1. El usuario ubica resistencias y fuentes en el lienzo vectorial (`QGraphicsScene`).
2. Se representan las conexiones entre terminales formando un Grafo $G = (V, E)$, donde:
 - V (Vértices/Nodos): Puntos de unión eléctrica.
 - E (Aristas/Ramas): Componentes eléctricos (Resistencias o Fuentes DC).

Paso 2: Clasificación Automática de Nodos

1. Se calcula el grado $d(v)$ (número de ramas conectadas) para cada vértice $v \in V$.
2. **Nodos Simples (Conexiones en Serie):** Si $d(v) = 2$, el nodo une únicamente 2 componentes en serie. Se etiquetan automáticamente con **letras minúsculas** ($a, b, c, \dots$).
3. **Nodos Principales / Esenciales (Bifurcaciones):** Si $d(v) \geq 3$, el nodo conecta 3 o más componentes. Se etiquetan con **letras mayúsculas** ($A, B, C, \dots$).

Paso 3: Identificación de Mallas e Inicialización de Corrientes ($I_1, I_2, \dots$)

1. **Criterio de Prioridad:** El sistema escanea las fuentes de voltaje V_1, V_2, V_3 , identifica la fuente con **mayor valor numérico de voltaje** ($V_{\max}$) y establece la orientación primaria de la primera malla (I_1) coincidiendo con el sentido convencional de salida de corriente de esta fuente (del polo positivo al negativo).
2. Se extrae una Base de Ciclos Fundamentales del grafo utilizando un árbol de expansión (*Spanning Tree*).
3. Se asigna una corriente de malla vectorial I_k orientada a cada ciclo linealmente independiente hallado.

Paso 4: Construcción del Sistema Matricial de Kirchhoff

Se construye automáticamente la ecuación matricial de mallas:

$$\mathbf{R} \cdot \mathbf{I} = \mathbf{V}$$

Donde:

- $\mathbf{R}_{k,k}$ (Diagonal): Suma total de resistencias en la malla k .
- $\mathbf{R}_{k,j}$ (Fuera de la diagonal): Suma de resistencias compartidas entre la malla k y la malla j (con signo negativo si las corrientes I_k e I_j atraviesan el elemento en sentidos opuestos).
- $\mathbf{V}_k$: Suma algebraica de las fuentes de tensión contenidas en la malla k .

Paso 5: Resolución y Cálculo de Propiedades por Componente

1. Se resuelve el vector de corrientes de malla: $\mathbf{I} = \mathbf{R}^{-1} \cdot \mathbf{V}$.
2. Para cualquier resistencia R_x seleccionada por el usuario:
 - **Corriente** (I_{R_x}): Suma vectorial de las corrientes de malla que la atraviesan.
 - **Caída de Tensión** (V_{R_x}): Calculada por Ley de Ohm ($V_{R_x} = I_{R_x} \cdot R_x$).
 - **Potencia Disipada** (P_{R_x}): $P_{R_x} = V_{R_x} \cdot I_{R_x}$.

Paso 6: Renderizado UI y Sección de Matrices

La interfaz despliega en el panel inferior las matrices $\mathbf{R}$, $\mathbf{I}$ y $\mathbf{V}$ dinámicamente actualizadas. Al hacer clic sobre cualquier R_x en el lienzo, un panel flotante/lateral inspecciona sus parámetros.

4 Leyes Físicas Aplicadas en la Parte Matemática

A. Ley de Ohm

Establece la relación lineal entre la corriente eléctrica I , la tensión V y la resistencia R :

$$V = I \cdot R$$

B. Leyes de Kirchhoff (KVL y KCL)

1. **Ley de Tensiones de Kirchhoff (LVK / KVL):** La suma algebraica de las diferencias de potencial eléctrico en cualquier malla o trayectoria cerrada es igual a cero:

$$\sum_{k=1}^n V_k = 0$$

2. **Ley de Corrientes de Kirchhoff (LCK / KCL):** La suma de las corrientes que entran a un nodo es igual a la suma de las corrientes que salen de él:

$$\sum I_{\text{entrantes}} = \sum I_{\text{salientes}}$$

5 El Rol del Álgebra Lineal en el Procedimiento

El Álgebra Lineal es el motor matemático central que permite transformar un circuito eléctrico dinámico en un problema numérico computable.

1. **Sistemas de Ecuaciones Lineales Simultáneas:** El análisis por mallas produce un conjunto de m ecuaciones independientes con m incógnitas (corrientes de malla $I_1, I_2, \dots, I_m$).
2. **Estructura Matricial:** El sistema se plantea como:

$$\begin{bmatrix} R_{11} & R_{12} & \dots & R_{1m} \\ R_{21} & R_{22} & \dots & R_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ R_{m1} & R_{m2} & \dots & R_{mm} \end{bmatrix} \begin{bmatrix} I_1 \\ I_2 \\ \vdots \\ I_m \end{bmatrix} = \begin{bmatrix} V_1 \\ V_2 \\ \vdots \\ V_m \end{bmatrix}$$

3. **Inversión y Solución Numérica:** Mediante algoritmos de eliminación de Gauss-Jordan o descomposición LU (implementados eficientemente en `numpy.linalg.solve`), el sistema determina en $\mathcal{O}(n^3)$ el valor exacto de las corrientes en microsegundos, asegurando una aplicación ágil y optimizada.

6 Usabilidad, Accesibilidad e Interfaz de Usuario

– Accesibilidad (a11y):

- Alto contraste visual para esquemáticos.
- Atajos de teclado para agregar resistencias (R), fuentes (V) y borrar elementos (Delete).
- Indicadores visuales por código de colores para nodos minúsculos/mayúsculos.

– Diseño Responsive / Layout:

- División mediante `QSplitter`: Lienzo interactivo superior ajustable y visualizador matricial matemático inferior con capacidad de scroll.